TECHNISCHE HOCHSCHULE MITTELHESSEN
CAMPUS FRIEDBERG
MND
Mathematik, Naturwissenschaften und Datenverarbeitung

# Vorlesungsskript

## Fortgeschrittene Datenbanktechnologien

im Studiengang Master Wirtschaftsinformatik

über das Thema

# Temporale Datenbanken

von

Aleksandra Muskaj

Dozent: Prof. Dr. habil. Markus Siepermann

**Inhaltsverzeichnis**

1. Einleitung
2. Definition und Anwendung
   2.1 Anwendung
   2.2 Vergleich mit herkömmlichen Datenbanken
   2.3 Vorteile und Herausforderungen
   2.4 Bisherige Entwicklungen
3. Zeit in Temporalen Datenbanken
   3.1 Temporale Datentypen
   3.2 Zeit Dimensionen und Dimensionale -Beziehungen
   3.3 Beispiel
4. Temporale Datenbanken arten
   4.1 Snapshot-Datenbanken
   4.2 Rollback-Datenbanken
   4.3 Historische Datenbanken
   4.4 Temporale Datenbanken(Bi-temporale)
5. Temporale Datenbankmodellen
6. Operationen in temporalen Datenbanken mit SQL11
   6.1 Anwendungszeitraumstabellen
   6.2 System versionierte Tabellen
   6.3 Bi-temporaleTabellen
7. References

# 1. Einleitung

Zeit ist ein fundamentales Element unseres Lebens und spielt auch in vielen Datenbankanwendungen eine wichtige Rolle. Da die meisten Datenbankanwendungen von temporärer Natur sind, ist ein Verständnis der zeitlichen Aspekte entscheidend, um effektive und präzise Datenverwaltung und -abfragen zu ermöglichen.

Die vorliegende Vorlesungsskript behandelt das Thema der temporalen Datenbanken. Temporale Datenbanken stellen eine Erweiterung herkömmlicher Datenbanken dar, die es ermöglicht, zeitbezogene Informationen effektiv zu speichern und abzufragen. In dieser Einführung werden wir uns mit den Grundlagen temporaler Datenbanken befassen, ihre Konzepte und Anwendungen kennenlernen und einen Vergleich zu herkömmlichen Datenbanken ziehen. Wir werden auch die Vorteile und Herausforderungen von temporalen Datenbanken untersuchen sowie einen Überblick über bisherige Entwicklungen auf diesem Gebiet geben. Des Weiteren werden wir uns mit den verschiedenen temporalen Datenbankmodellen auseinandersetzen und die wichtigsten Operationen in temporalen Datenbanken mit SQL Server kennenlernen.

# 2. Definition und Anwendung

Viele Unternehmen bieten Produkte an, deren Preise sich im Laufe der Zeit ändern. Energieversorgungsunternehmen bieten oft unterschiedliche Tarife für verschiedene Tageszeiten an. In vielen Städten variieren die Parkgebühren je nach Tageszeit. Einige Restaurants bieten spezielle Mittagsangebote nur für bestimmte Zeiten des Tages. Daher sind die Preise in diesen Beispielen zeitabhängig und werden typischerweise in Tabellen mit Preisen in Verbindung mit einem Zeitraum zusammengefasst.

Die meisten Datenbankanwendungen sind von temporärer Natur, zum Beispiel Finanzanwendungen wie Portfoliomanagement, Buchhaltung und Bankwesen, Medizinakten und Lagerverwaltung, Terminplanungsanwendungen wie Flug-, Zug- und Hotelreservierungen sowie Projektmanagement und wissenschaftliche Anwendungen wie Wetterüberwachung (Jensen 1999).

In vielen Fällen reicht es, nur den aktuellen gültigen Wert in einem Datenbestand zu speichern und bei Änderungen den alten Wert zu überschreiben. Wenn jedoch die Anforderung besteht, alle Änderungen zu dokumentieren, ist eine temporale Datenhaltung erforderlich.

Neben der Aufzeichnung, wann bestimmte Datenwerte gültig sind, möchte man möglicherweise (aus Gründen der Sicherung oder Analyse) auch historische Aufzeichnungen über Änderungen in der Datenbank speichern. Jedes Mal, wenn eine Änderung in einer Datenbank vorgenommen wird, kann das System automatisch einen Transaktionszeitstempel speichern.

„Eine temporale Datenbank ist eine Datenbank, die Funktionen zur Unterstützung der Speicherung, Abfrage und Manipulation von zeitabhängigen Daten umfasst.“ (C.J. Date, Hugh Darwen, Nikos Lorentzos 2002)

Temporale Datenbanken bieten eine Vielzahl von temporalen Datentypen und speichern Informationen, die sich auf zeitliche Variationen beziehen - Vergangenheit, Gegenwart und Zukunft. Beispiele hierfür sind die Historie des Aktienmarkts oder die Bewegungen von Mitarbeitern innerhalb einer Organisation. Eine temporale Datenbank speichert also eine Sammlung von zeitbezogenen Daten.

Merkmale temporaler Datenbanken:

- bieten integrierte Unterstützung für die Zeitdimension.
- speichern Daten zu den Zeitaspekten.
- enthalten historische Daten anstelle aktueller Daten.
- bieten eine einheitliche Möglichkeit, mit historischen Daten umzugehen.

Temporale Daten werden erstellt, indem die normalen Daten mit Zeitstempeln versehen werden.

Zur Zeitstempelung der Einträge in Temporalen Datenbanken gibt es zwei Möglichkeiten:

• Tupelstempelung

• Attributstempelung

Bei der Tupelstempelung ist die Relationale Tabelle um zwei Spalten erweitert. Eine Spalte steht für die Startzeit, die zweite für die Endzeit. Von den oben vorgestellten Datentypen eignen sich für die Repräsentation die Zeitpunkte. Die Zeiten beziehen sich auf die gesamte Zeile, also auf die gesamte Entität und auf alle Werte des Tupels. Ebenso könnte man für jedes einzelne Attribut zwei zusätzliche Spalten für die Zeitstempelung einführen. Dies ist dann Sinnvoll wenn die Änderungsfrequenz einzelner Objekte sehr groß ist, führt aber zu vielen Spalten, weshalb die Attributstempelung nicht verfolgt wird. (Mutschler 2019)

Das allgemeine Konzept ist die Erweiterung der relationalen Datenbank um eine Zeitdimension. Vereinfachend lassen sich Relationen in TDB als dreidimensionale Objekte betrachten. Die ersten beiden Dimensionen, Attribute und Werte, werden um eine dritte Dimension Zeit erweitert. D.h. jedem Tupel einer Relation wird eine bestimmte Zeit zugeordnet in der es existiert bzw. gültig ist.

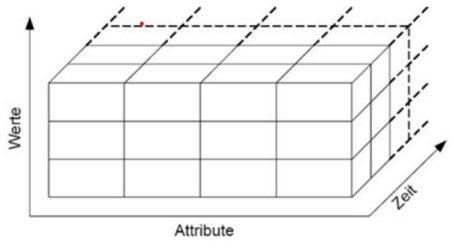


Abbildung 1: Temporale Relationen als dreidimensionales Objekt [1]

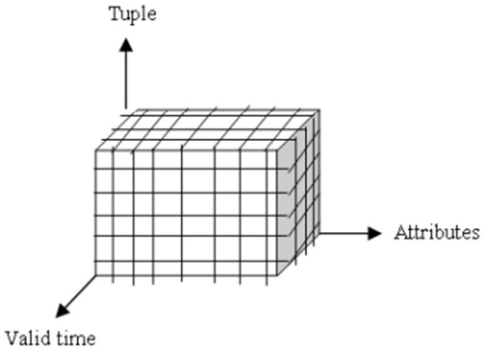


Abbildung 2: Kubisches Model[2]

Das folgende Beispiel visualisiert, wie Daten in einer temporären Tabelle gespeichert werden. Die in der Abbildung dargestellte Tabelle stellt einen aktuellen Stand(Snapshot) der Datenbank dar und die dritte Achse für Zeit ist hinzugefügt.

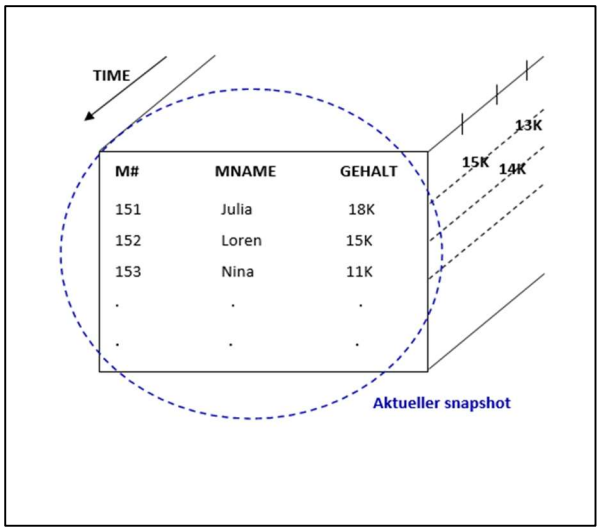


Abbildung 3: Temporal Dimension

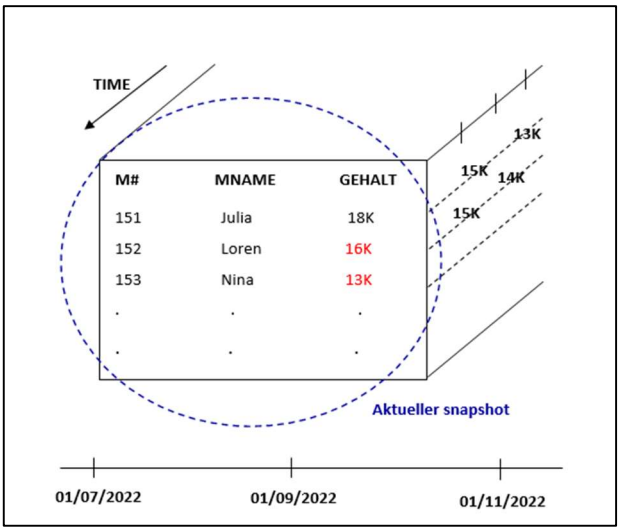


Abbildung 4 Temporal Dimension mit Änderungen

[1] TAOU HASSANI: Konzepte temporaler Datenbanken. 2004. – TAOU HASSANI: Konzepte temporaler Datenbanken. 2004. – abgerufen 01. Juni 2023

[2] https://www.kau.edu.sa/files/830/researches/55976_26285.pdf

Julia würden am 1. Juli befördert und hat eine Gehaltserhöhung bekommen und später in Abb. 4 Ninas Gehalt wurde korrigiert . Nicht nur die Werte werden korrigiert sondern Änderungen werden in Temporale Datenbanken erfasst und sind jederzeit aufrufbar.

### 2.1 Anwendung

Zu den Anwendungsfällen für temporale Tabellen gehören:

- Prüfung aller Datenänderungen und Durchführung von Datenforensik bei Bedarf
- Rekonstruktion des Datenstandes zu einem beliebigen Zeitpunkt in der Vergangenheit
- Berechnen von Trends im Zeitverlauf
- Beibehaltung einer sich langsam verändernden Dimension für Anwendungen zur Entscheidungsunterstützung
- Wiederherstellung nach versehentlichen Datenänderungen und Anwendungsfehlern

Abbildung 5 Temporal Dimension

Die Anwendungsmöglichkeiten von temporalen Datenbanken in der Praxis sind vielfältig. Diese Datenbanken sind besonders notwendig in den Wirtschaftswissenschaften, z.B. für Prognosen, Analysen, Strategieplanung, Rechnungsberichte, Anwendungen mit zeitlichen Aspekten, Buchhaltungssysteme, Transaktionsüberwachung, sowie in den Gebieten von ”Data Warehouse” und ”Business Itelligence”. Außerdem sind sie wichtig für die Untersuchung von historischen und medizinischen, sowie von Verkehrsdaten (Osipov 2017).

Typische Anwendungsfälle und Szenarien für zeitliche Datenbanken umfassen zeitabhängige wirtschaftliche Daten wie:

- Unternehmensgewinne, Zinssätze

- Aktienkurse und Wechselkurse

Typische informell formulierte Abfragen könnten sein:

- Gib mir die Historie des Dollar-Pfund-Sterling-Wechselkurses des letzten Monats.

- Gib mir die Aktienkurse der Tesla am 17. Oktober 1996.

Bei medizinischen Diagnosen zieht man oft Rückschlüsse aus der Krankheitsgeschichte eines Patienten, d. h. aus der Entwicklung seiner Erkrankung. Letztere wird durch eine Reihe von Werten be-

schrieben, wie z. B. Körpertemperatur, Cholesterinkonzentration im Blut, Blutdruck usw. Wie im ersten Beispiel ist jeder dieser Werte nur während eines bestimmten Zeitraums gültig (z. B. an einem bestimmten Tag). Ein Arzt würde typischerweise die Wertehistorie eines Patienten abrufen, Trends analysieren und Diagnosen aufgrund solcher Beobachtungen stellen (Temporal tables 2023).

### 2.2 Vergleich mit herkömmlichen Datenbanken

Der Hauptunterschied zwischen temporalen und nicht-temporalen Daten besteht darin, dass zeitliche Daten in der Datenbank mit einer zeitlichen Einschränkung versehen sind, die angibt, wann sie gültig sind oder in der Datenbank gespeichert werden. In konventionellen Datenbanken wird davon ausgegangen, dass Daten zum aktuellen Zeitpunkt "jetzt" gültig sind. Wenn die Daten in temporalen Datenbanken aktualisiert, gelöscht oder eingefügt werden, wird der Zustand der temporalen Datenbank überschrieben und ein neuer Zustand wird festgelegt. Der Zustand vor Änderungen an der Datenbank steht nach der Aktualisierung der Daten nicht mehr zur Verfügung. Durch die Verknüpfung von Zeit mit Daten ist es möglich, die verschiedenen Zustände der Datenbank zu speichern.

| ***Konventionelle Datenbanken:*** | ***Temporale Datenbanken:*** |
|---|---|
| - bieten nur eine Momentaufnahme der Daten<br>- Änderungen werden als Modifikationen am Zustand betrachtet.<br>Zeitliche Entwicklung wird nicht unterstützt<br>- Keine Informationen über die Vergangenheit<br>-Analyse von Zeitdaten erschwert | - Inkorporieren das Konzept der Zeit und behalten historische Informationen bei.<br>- Änderungen werden als Hinzufügungen zu den in der Datenbank gespeicherten Informationen betrachtet.<br>- Effizienter Zugriff auf vergangene Zustände.<br>- Komplexe Schema |

### 2.3 Vorteile und Herausforderungen

Die Daten in einer temporalen Datenbank werden nicht nur gespeichert, sondern auch im Laufe der Zeit verfolgt und verwaltet. Diese Art von Datenbank wird in verschiedenen Bereichen verwendet, darunter Finanzen, Gesundheitswesen, Transport und wissenschaftliche Forschung. Einer der bedeutenden Vorteile der Verwendung temporaler Datenbanken besteht darin, dass sie die Speicherung von historischen Daten ermöglichen, die für Analysen und Entscheidungsfindung verwendet werden können.

Ein weiterer Vorteil der Verwendung temporaler Datenbanken besteht darin, dass sie Abfragen zu einem bestimmten Zeitpunkt ermöglichen. Benutzer können die Datenbank abfragen, um Daten ab-

zurufen, wie sie zu einem bestimmten Zeitpunkt existierten. Diese Funktion ist besonders nützlich für Prüfungen und Compliance. Im Finanzsektor können temporale Datenbanken beispielsweise verwendet werden, um Transaktionen, Kontostände und andere finanzielle Daten zu verfolgen. Diese Daten können zur Einhaltung von Vorschriften dienen und potenzielle Betrugsfälle oder Fehler identifizieren.

Zusammenfassend sind temporale Datenbanken ein wertvolles Werkzeug zur Speicherung, Verfolgung und Verwaltung zeitlich veränderlicher Daten. Die Vorteile der Verwendung temporaler Datenbanken umfassen die Speicherung historischer Daten, Abfragen zu einem bestimmten Zeitpunkt und die einfache Verwaltung von zeitlichen Daten. Diese Vorteile machen temporale Datenbanken zu einem wertvollen Werkzeug in verschiedenen Bereichen, einschließlich Finanzen, Gesundheitswesen, Transport und wissenschaftlicher Forschung.

**Herausforderungen**

1. Mehr Speicherbedarf: Temporale Datenbanken erfordern in der Regel mehr Speicherplatz im Vergleich zu herkömmlichen Datenbanken. Dies liegt daran, dass sie zusätzliche Informationen zur Verfolgung der Zeitkomponente enthalten, wie z.B. Zeitstempel oder Gültigkeitsintervalle. Dadurch erhöht sich der Speicherbedarf für die Datenbank.

2. Komplexität bei der Abfrageverarbeitung: Die Verarbeitung von Abfragen in temporalen Datenbanken kann komplex sein. Da die Daten über die Zeit hinweg verfolgt und verwaltet werden, müssen komplexe Anfragen gestellt werden, um spezifische Zeitpunkte oder Zeitintervalle abzurufen. Dies kann zu komplexen Abfragesyntaxen und längeren Ausführungszeiten führen.

3. Komplexität der Wartung einschließlich Sicherung und Wiederherstellung: Die Wartung einer temporalen Datenbank kann aufgrund der zusätzlichen Zeitkomponente komplexer sein. Die Sicherung und Wiederherstellung von Daten in temporalen Datenbanken erfordert spezielle Verfahren, um sowohl die aktuellen Daten als auch die historischen Daten wiederherzustellen. Dies erfordert zusätzliche Ressourcen und kann zeitaufwändig sein.

4.Schema Design: Das Schema der temporalen Datenbanken muss die Zeitdimension berücksichtigen. Die Erstellung eines solchen Schemas ist schwieriger als die Erstellung eines Schemas für nicht-temporale Datenbanken.

## 2.4 Bisherige Entwicklungen

Die Ergebnisse mehrerer Jahrzehnte Forschung in Bezug auf temporale Abfragesprachen zeigen, dass es tatsächlich möglich ist, die Unterstützung für das zeitliche Datenmanagement in Abfragesprachen zu integrieren, so dass Aussagen wie die oben genannte einfacher formuliert werden können.

Eine temporale Abfragesprache ist eine Datenbankabfragesprache, die eine eingebaute Unterstützung für die Abfrage und Modifikation von zeitbezogenen Daten bietet und es ermöglicht, Aussagen und Einschränkungen für solche Daten zu spezifizieren. Eine temporale Abfragesprache ist in der Regel eng mit einem temporalen Datenmodell verbunden, das die zugrunde liegenden Datenstrukturen definiert, auf die sich die Sprache bezieht (C. S. Snodgrass 2009).

Seit Mitte der achtziger Jahre, wurde eine Vielzahl von verschiedenartiger Sprachvorschlägen vorgestellt: IXSQL, TSQL2, SQL/TP, um einige zu nennen. Mit wenigen Ausnahmen sind diese Sprachvorschläge Erweiterungen zu SQL2 und basieren auf dem relationalen Datenmodell.

SQL-86 und SQL-89 beinhalteten noch keine Möglichkeit temporale Daten zu verwalten. Mit SQL-92 (SQL2) wurden die Datentypen DATETIME und INTERVAL eingeführt. D.h. der erste Schritt in Richtung Zeitunterstützung wurde gemacht. Bald wurde jedoch erkannt, dass allein diese zwei Datentypen nicht ausreichen, um die Erfordernisse einer temporalen DBMS-Sprache abzudecken.

Im Juni 1993 formte sich nach einem "Workshop on an Infrastructure for Temporal Databases" in Arlinton, Texas ein TSQL2 Komitee mit dem Ziel, eine Sprachspezifikation einer temporalen Anfragesprache zu verfassen. Im folgenden Januar wurde eine erste Version veröffentlicht und die endgültige Version folgte im September 1994. In dieser Sprache ist ein Zeitstempelattribut hinzugefügt, das Mengen von bitemporalen Chronons enthält, und zwar die Zeitpunkte, wann ein Fakt gültig war und wann er eingetragen wurde. TSQL2 unterstützt die Evolution der relevanten Objekte und damit der ihnen zugeordneten Daten im Zeitablauf (Hassani 2004).

In SQL2 werden bereits einige zeitorientierte Datentypen angeboten:

- DATE
- TIME
- TIMESTAMP als Kombination der beiden erstgenannten
- INTERVAL ist für die Zeitdauer (nicht zu Verwechseln mit Zeitintervallen, die durch zwei Zeitpunkte begrenzt sind!)

- PERIOD für den Zeitintervall-Datentyp: PERIOD(datetime1, datetime2)
- INTERVAL(PERIOD): Umwandlung einer Periodenangabe in eine Zeitdauer.
- INTERSECT(PERIOD_1, PERIOD_2): Ermittlung der Schnittmenge zweier Perioden
- BEGIN(PERIOD) für den Anfangszeitpunkt einer Periode
- END(PERIOD) für den Endzeitpunkt einer Periode
- ALTER TABLE: Zeitdimensionen nachträglich einzufügen, oder zu entfernen.

Spezielle Werte zur Definition Periodenbegrenzung

- 'beginning' steht für unendlich weit zurückliegende Vergangenheit.
- 'forever' steht für unendlich weit entfernte Zukunft.
- 'now' steht für die aktuelle Zeit.
- PERIOD [beginning,- forever]' stellt den maximal in einem System abbildbaren Zeitraum dar.

Vergleichs-Operatoren

- Gleichheits-Operator: (=)
- PRECEDES : die Vorzeitigkeit einer Periode gegenüber einer anderen Periode ausdrücken. Bsp.: X PRECEDES Y, dann ist END(X)<< BEGIN(Y);
- MEETS ist ein Spezialfall von PRECEDES, bei dem END(X) und BEGIN(Y) genau ein Chronon auseinander liegen.
- OVERLAPS: da wird ermittelt, ob die Schnittmenge zweier Perioden nicht leer ist.
- CONTAINS ist wiederum ein Spezialfall, bei dem eine Periode vollständig in einer anderen Periode enthalten sein muss.

SQL:2011 erweiterte die Sprache um temporale Abfragemöglichkeiten. Dies umfasst die Möglichkeit, Abfragen zu bestimmten Zeitpunkten oder innerhalb bestimmter Zeiträume zu stellen und historische Daten zu untersuchen. Tabellendefinitionen wurden erweitert, um "Anwendungszeitraumstabellen" (gültigkeitszeitabhängige Tabellen) und "system-versionierte Tabellen" (transaktionszeitabhängige Tabellen) zu definieren.

**Unterstützung in aktuellen DBMS-Produkten**

Momentan gibt es kein Datenbankmanagementsystem, das den temporalen Standard komplett unterstützt. Allerdings sind eine Reihe von Systemen, die über temporale Features verfügen und einen Teil des Standards umsetzen (Mutschler 2019):

• Oracle (ab Version 12): Temporale Funktonen von SQL:2011

• IBM DB2 (ab V10): Temporale Daten, basierend auf SQL:2011

• PostgreeSQL (ab Version 9.2): Temporale Features

• Teradata (ab Version 14): TSQL2 Features

• Micosoft SQL Server (ab Version 16): Temporale Tabellen

• MariaDB (ab Version 10.3): Temporale Funktionen

## 3. Zeit in Temporalen Datenbanken

Wenn wir über Zeit als diskrete Einheiten nachdenken, können je nach Zweck oder Anwendung unterschiedlich große Einheiten angemessen sein. Das Alter von Vulkanen kann in Jahren, Jahrzehnten oder sogar Jahrhunderten gemessen werden. Das Alter von Babys kann in Jahren, Monaten, Wochen oder Tagen gemessen werden. Das Alter von Bakterien wird in Sekunden oder Millisekunden gemessen. Die Größe der Zeiteinheiten, die in einem bestimmten Szenario verwendet werden, wird als Granularität der temporalen Einheiten bezeichnet. Unabhängig von Zeit Granularität, verschiedenen Informationsarten können unterschiedliche Zeitfolgen haben. Im Allgemeinen kann das Verhalten zeitlicher Einheiten in eine von vier Grundkategorien eingeteilt werden, nämlich:

- Diskret
  Diskrete Datenattribute werden zu bestimmten Zeitpunkten aufgezeichnet, haben aber zu keinem anderen Zeitpunkt eine Definition. Diskrete Daten werden einzelnen Ereignissen zugeordnet, beispielsweise „Eine vollständige Untersuchung fand an einem bestimmten Datum statt“.
- Kontinuierlich
  Kontinuierliches Verhalten wird beobachtet, wenn ein Attributwert im Laufe der Zeit konstant aufgezeichnet wird, sodass sich sein Wert ständig ändern kann. Kontinuierliches Verhalten findet sich häufig in Überwachungssystemen, die kontinuierliche Kenngrößen erfassen – beispielsweise einem Tachometer eines Autos.
- Zeitraumbasiert
  Periodenbasierte Daten modellieren das Verhalten von Ereignissen, die über einen bestimmten Zeitraum auftreten, am Ende dieses Zeitraums jedoch undefiniert werden. Ein Beispiel für

periodenbasierte Daten wären Aufzeichnungen über den Drogenkonsum von Patienten, bei denen ein Patient ein Medikament über einen vorgeschriebenen Zeitraum einnimmt und es dann nicht mehr einnimmt.

Abbildung 6[3]

Abbildung 7

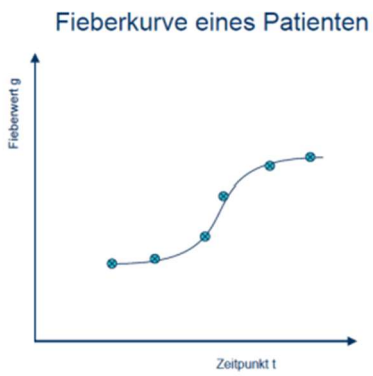


Abbildung 8

### 3.1 Temporale Datentypen

Zur Zeitstempelung bieten sich explizit Temporale Datentypen an, da sie von sich aus eine zeitliche Bedeutung besitzen und somit vor Fehlinterpretationen geschützt sind. Beim Versuch, Zeit darzustellen und darüber zu argumentieren, gibt es vier wichtige Konzepte.

• Punkte: Formal hat ein Zeitpunkt keine Dauer; er bezieht sich einfach auf eine bestimmte Position in der diskutierten Zeitleiste. Wir können über den Zeitpunkt sprechen, an dem ein Ereignis beginnt oder endet.

• Menge von Zeitpunkten

• Dauer: Eine Dauer bezieht sich auf eine Anzahl von Zeitquanten; zum Beispiel sind eine Woche, zwei Monate, drei Jahre und 10 Sekunden allesamt Dauern. Eine Dauer bezieht sich auf einen bestimmten Umfang (Größe) eines Teils einer Zeitleiste, aber nicht auf die Richtung (egal, ob wir von vor einer Woche oder von einer Woche ab drei Tagen in der Zukunft sprechen, wir beziehen uns immer auf eine Zeitspanne von einer Woche).

[3] HANNO EICHELBERGER AND ALEXANDER RUDER AND KHALID ADDAKIRI: Temporale Datenbanken. 2005

• Intervall: Ein Intervall hat einen Startzeitpunkt und einen Endzeitpunkt. Unter Verwendung einer formalen Notation können wir uns auf ein Intervall I(s, e) mit dem Startpunkt 's' und dem Endpunkt 'e' beziehen, und alle Zeitpunkte von s bis e (einschließlich) bilden das Intervall. Beachten Sie, dass die Annahme (Einschränkung) besteht, dass der Zeitpunkt 's' nicht nach dem Zeitpunkt 'e' liegt (ein Intervall von null Zeitquanten hätte einen Startpunkt und einen Endpunkt, die gleich sind).

## 3.2 Zeit Dimensionen und Dimensionale -Beziehungen

### Zeit Dimensionen

In temporalen Datenbanken gibt es verschiedene Zeitdimensionen, die verwendet werden können, um die Zeit darzustellen. Die beiden häufigsten Zeitdimensionen sind gültige Zeit (valid time) und Transaktionszeit (transaction time).

- Gültige Zeit (Valid Time): Die gültige Zeit einer Tatsache bezieht sich auf den Zeitraum, in dem die Tatsache in der modellierten Realität wahr ist. Sie wird normalerweise vom Benutzer angegeben und kann sich ändern. Die gültige Zeit ermöglicht es, historische Informationen abzurufen und die Vergangenheit in einer temporalen Datenbank zu analysieren. Ein gültiger Zeitraum kann sowohl in der Vergangenheit liegen, die aktuelle Zeit umfassen oder in der Zukunft liegen.

- Transaktionszeit (Transaction Time): Die Transaktionszeit einer Tatsache bezieht sich auf den Zeitpunkt, zu dem die Information über ein modelliertes Objekt oder Ereignis in der Datenbank gespeichert wird. Die Transaktionszeit wird automatisch vom Datenbankmanagementsystem (DBMS) erfasst und kann nicht geändert werden. Sie ermöglicht die Nachverfolgung von Änderungen und den Aufbau einer Historie basierend auf den Zeitstempeln der Transaktionszeiten. In einem Transaktionszeitraum können nur Zeiträume in der Vergangenheit oder bis zur aktuellen Zeit vorkommen. In einer Transaktionszeittabelle werden Einträge niemals gelöscht. Es können nur neue Einträge hinzugefügt werden, und bestehende Einträge können aktualisiert werden, indem ihre Transaktionsendzeit gesetzt wird, um anzuzeigen, dass sie nicht mehr aktuell sind.

-Benutzerdefinierte Zeit (User-Defined Time) ist eine weitere Zeitdimension, die in temporalen Datenbanken verwendet werden kann. Im Gegensatz zur gültigen Zeit und Transaktionszeit, bei denen die Zeitinformation entweder vom Benutzer angegeben oder automatisch vom System erfasst wird, ermöglicht die benutzerdefinierte Zeit dem Benutzer, eine eigene Zeitdimension zu definieren, die spezifisch für seine Anforderungen ist (R. Snodgrass 2000).

Benutzerdefinierte Zeit ist notwendig, wenn zusätzliche zeitliche Informationen, die nicht durch Transaktions- oder Gültigkeitszeit behandelt werden, in der Datenbank gespeichert werden. Die Werte der benutzerdefinierten zeitlichen Attribute werden vom DBMS nicht interpretiert und sind daher am einfachsten zu unterstützen. Als Beispiel für benutzerdefinierte Zeit betrachten wir eine Promotion-Tabelle mit drei Attributen: Name, Rang und Genehmigungsdatum. Das Genehmigungsdatum (eine benutzerdefinierte Zeit) ist das Datum, an dem das Beförderungskomitee die Beförderung genehmigt hat; die Gültigkeitszeit ist das Datum, an dem die Beförderung wirksam wird; und die Transaktionszeit ist das Datum, an dem die Informationen zur Beförderung in der Datenbank gespeichert wurden (Richard Snodgrass 1986).

**Dimensionale -Beziehungen**

Uni-Temporale Relationen: Sie haben eine Zeitachse, entweder Gültigkeitszeit oder Transaktionszeit.

Bi-Temporale Relationen: Sie haben beide Zeitachsen - Gültigkeitszeit und Transaktionszeit. Sie umfassen Gültigen Startzeitpunkt, Gültigen Endzeitpunkt, Transaktionsstartzeitpunkt und Transaktionendzeitpunkt (Sam 2020).

Gültige Zeit und Transaktionszeit gelten als die häufigsten Zeitmodelle in einer temporalen Datenbank und werden als Zeitdimensionen bezeichnet. In einigen Anwendungen wird nur eine der Dimensionen benötigt, während in anderen Fällen beide Zeitdimensionen erforderlich sind, was zu einer zweizustandigen Zeit (bitemporal-state time) führt.

### 3.3 Beispiel

Um die Gültige Zeit und Transaktionszeit, praktisch zu verstehen, nehmen wir das folgende Beispiel (bearbeitet nach dem Model in (Sam 2020)):

Laura Müller wurde am 1. Mai 2001 in Erfurt geboren. Laura Vater meldete die Geburt seiner erstgeborenen Tochter am 3. Mai 2001 im Rathaus an. Laura wuchs auf, ging zur Schule und schloss ihr Studium 2022 in Erfurt. Nach dem Abschluss zog sie in ihre eigene Wohnung nach Köln. Obwohl sie am 26. August 2022 ausgezogen war, vergaß sie offiziell die Adressänderung zu registrieren. Erst zum Jahreswechsel erinnerte sie ihre Mutter daran, dass sie sich registrieren lassen müsse, was sie schließlich am 27. Dezember 2020 tat. Obwohl Laura eine vielverspre-

chende Zukunft vor sich hatte, endet ihre Geschichte tragisch. Laura Müller wurde am 1. April 2023 versehentlich von einem Lastwagen erfasst. Der Gerichtsmediziner gab ihren Tod noch am selben Tag bekannt.

Wichtige Zeitpunkte:

- Laura wurde am 1. Mai 2001 in Erfurt geboren.
- Ihr Vater registrierte ihre Geburt zwei Tage später am 3. Mai 2001.
- Sie zog 26. August 2022 nach Köln.
- Sie meldete ihre Adresswechsel erst am 27. Dezember 2022.
- Laura ist 1. April 2023 gestorben.

Um das Leben von Laura Müller in einer aktuellen (nicht-temporalen) Datenbank zu speichern, verwenden wir eine Tabelle Person (Name, Adresse). (Um es zu vereinfachen, ist der Name als Primärschlüssel der Person definiert.)

| **Datum** | **Ereignis** | **Database Action** | **Zustand in der Datanbank** |
|---|---|---|---|
| **1. Mai 2001** | Laura wird geboren | - | Es gibt keine Person namens Laura Müller |
| **3. Mai 2001** | Lauras Vater registrierte ihre Geburt | Inserted:Person(Laura Müller, Erfurt) | Laura Müller lebt in Erfurt |
| **26. August 2022** | Laura zieht um. | - | Laura Müller lebt in Erfurt |
| **27. Dezember 2022** | Laura registriert ihre neue Adresse. | Updated:Person(Laura Müller, Köln) | Laura Müller lebt in Köln |
| 1. April 2023 | Laura ist gestorben. | Deleted:Person(Laura Müller) | Es gibt keine Person namens Laura Müller |

An dem Tag, an dem Lauras Vater ihre Geburt registriert, fügt ein Beamter den folgenden Eintrag in die Datenbank ein: Person(Laura Müller, Erfurt). Beachten Sie, dass das Datum selbst nicht in der Datenbank gespeichert wird.

Nach ihrem Abschluss zieht Laura um, vergisst jedoch, ihre neue Adresse zu registrieren. Lauras Eintrag in der Datenbank wird erst am 27. Dezember 2022 geändert, als sie sich endlich meldet. Ein Beamter aus Köln aktualisiert ihre Adresse in der Datenbank. Die Tabelle "Person" enthält jetzt Per-

son(Laura Müller, Köln). Beachten Sie, dass die Information, dass Laura in Erfurt lebt, überschrieben wurde, sodass es nicht mehr möglich ist, diese Information aus der Datenbank abzurufen. Ein Beamter, der am 28. Dezember 2022 auf die Datenbank zugreift, würde erfahren, dass Laura in Köln lebt. Technisch betrachtet: Wenn ein Datenbankadministrator am 26. Dezember 2022 die Abfrage SELECT ADDRESS FROM PERSON WHERE NAME='Laura Müller' ausführen würde, wäre das Ergebnis Erfurt. Wenn dieselbe Abfrage zwei Tage später ausgeführt wird, lautet das Ergebnis Köln. Bis zu seinem Tod würde die Datenbank angeben, dass sie in Köln lebt. Am 1. April 2023 löscht der Gerichtsmediziner den Eintrag von Laura Müller aus der Datenbank. Danach würde die Ausführung der oben genannten Abfrage kein Ergebnis mehr liefern.

**Uni-Temporale Relationen(Gültigkeitszeit)**

Für das obige Beispiel werden zur Aufzeichnung der Gültigkeitszeit zwei Felder zur Personentabelle hinzugefügt: Gültig-von und Gültig-bis. Diese geben den Zeitraum an, in dem die Adresse einer Person in der realen Welt gültig ist. Am 3. Mai 2001, wenn Laura angemeldet wurde, trägt ein Beamter einen neuen Eintrag in die Datenbank ein, in dem steht, dass Laura seit dem 1. Mai in Erfurt lebt. Die Daten werden zwar am 3. eingetragen, die Datenbank angibt jedoch, dass die Informationen seit dem 1. gültig sind. Der Beamte weiß noch nicht, ob oder wann Laura an einen anderen Ort ziehen wird, daher wird das Feld Gültig-bis auf Unendlich (∞) gesetzt. Der Eintrag in der Datenbank lautet:

Person(Laura Müller, Erfurt, 1. Mai 2001, ∞).

Am 27. Dezember 2020 gibt Laura ihre neue Adresse in Köln an, wo sie seit dem 26. August 2022 wohnt. Es wird ein neuer Datenbankeintrag erstellt, um diese Tatsache festzuhalten:

Person(Laura Müller, Köln, 26. Aug 2022, ∞).

Der ursprüngliche Eintrag Person Person(Laura Müller, Erfurt, 1. Mai 2001, ∞) wird nicht gelöscht, sondern das Gültig-bis-Attribut wird aktualisiert, um anzuzeigen, dass bekannt ist, dass Laura am 25. August 2022 aufgehört hat, in Erfurt zu leben. Die Datenbank enthält jetzt zwei Einträge für Laura Müller:

| Name | City | Valid From | Valid Till |
|---|---|---|---|
| **Laura** | Erfurt | 1. Mai 2001 | 25. Aug 2022 |
| **Laura** | Köln | 26. Aug 2022 | ∞ |

Wenn Laura stirbt, wird ihr aktueller Eintrag in der Datenbank aktualisiert und es wird angegeben, dass Laura nicht mehr in Köln lebt. Die Datenbank sieht jetzt wie folgt aus:

| Name | City | Valid From | Valid Till |
|---|---|---|---|
| **Laura** | Erfurt | 1. Mai 2001 | 25. Aug 2022 |
| **Laura** | Köln | 26. Aug 2022 | 1. Apr 2023 |

**Bi-Temporale Beziehung**

Um die zusätzlich Transaktionszeit im obigen Beispiel zu ermöglichen, werden zwei weitere Felder zur Person-Tabelle hinzugefügt: Transaktion eingetragen und Transaktion abgelöst. Transaktion eingetragen ist die Zeit, zu der eine Transaktion durchgeführt wurde, und Transaktion abgelöst ist die Zeit, zu der die Transaktion abgelöst wurde. Dadurch wird die Tabelle zu einer bi-temporalen Tabelle. Nun enthält die Datenbank vier zusätzliche Einträge: Gültig von, Gültig bis, Transaktion eingetragen und Transaktion abgelöst.

Der Datenbankinhalt sieht folgendermaßen aus:

Name, Stadt, Gültig von, Gültig bis, Eingetragen, Ablösung

Zunächst, als Lauras Vater ihre Geburt registriert, wäre der gültige Startzeitpunkt der 1. Mai 2001, ihr tatsächliches Geburtsdatum. Die Zeit, zu der die Transaktion eingetragen wird, wäre jedoch der 3. Mai 2001. Es wird ein neuer Datenbankeintrag erstellt, um diese Tatsache festzuhalten:

Person(Laura, Erfurt, 1. Mai 2001, ∞, 3. Mai 2001, ∞).

Ähnlich, wenn Laura ihre Adressänderung in Köln registriert, wird ein neuer Eintrag erstellt. Der gültige Startzeitpunkt für diesen Eintrag ist der 26. August 2022, das tatsächliche Datum, ab dem sie in Köln lebt. Die Zeit, zu der die Transaktion eingetragen wird, wäre jedoch der 27. Dezember 2022. Da es unklar ist, wie lange sie in Köln leben wird, wären das Transaktionsenddatum und das Gültigkeitsenddatum unendlich. Gleichzeitig wird der ursprüngliche Eintrag mit dem gültigen Enddatum und der Transaktion, die ihn ablöst, aktualisiert.

Am 27. Dezember 2022 gibt Laura seine neue Adresse in Köln an:

Person(Laura, Köln, 26. Aug 2022, ∞, 27. Dez 2022, ∞).

Der ursprüngliche Eintrag wird aktualisiert.

Person(Laura, Erfurt, 1. Mai 2001, 25. August 2022, 3. Mai 2001, 27. Dezember 2022).

Nun sieht die Datenbank ungefähr so aus:

| Name | City | Valid From | Valid Till | Entered | Superseded |
|---|---|---|---|---|---|
| **Laura** | Erfurt | 1. Mai 2001 | 25. August 2022 | 3. Mai 2001 | 27. Dezember 2022 |
| **Laura** | Köln | 26. August 2022 | ∞ | 27. Dezember 2022 | ∞ |

Die Datenbank zeichnet nicht nur auf, was in der realen Welt passiert ist, sondern auch, was zu verschiedenen Zeiten offiziell aufgezeichnet wurde.

Aufgabe : Wie wird der Eintrag nach Lauras Tod sein und wie wird die Datenbank dann aussehen?

## 4. Temporale Datenbanken arten

Es können vier Arten von Datenbanken identifiziert werden, je nachdem, ob eine Datenbank die Unterstützung für Gültigkeitszeit und/oder Transaktionszeit bietet und in welchem Maße die Datenbank in Bezug auf die Zeit aktualisiert werden kann.

### 4.1 Snapshot-Datenbanken

Die Snapshot-Tabelle in einer Datenbank enthält nur den aktuellen Status der Daten. Sie repräsentiert den Momentaufnahme-Zustand der Datenbank zu einem bestimmten Zeitpunkt, ohne historische Veränderungen oder vergangene Zustände zu berücksichtigen (Ekta 2018). Im Gegensatz zu temporalen Tabellen, die Informationen über die Vergangenheit und Änderungen im Laufe der Zeit speichern, enthält die Snapshot-Tabelle nur die aktuellen Daten.

Die Snapshot-Tabelle wird häufig verwendet, um einen schnellen Zugriff auf die aktuellen Informationen in der Datenbank zu ermöglichen. Sie bietet eine Momentaufnahme des aktuellen Zustands, ohne dass zusätzliche Zeitstempel oder Versionierungsinformationen berücksichtigt werden müssen. Dies kann nützlich sein, wenn nur die aktuellen Daten benötigt werden und historische Informationen nicht relevant sind oder nicht gespeichert werden müssen.

SNAPSHOT TABLE

| Emp_no | Salary | Name |
|---|---|---|
| Em_52 | 20000 | Smith |
| Em_53 | 15000 | Jones |

Abbildung 9 [4]

Obschon nicht-temporale Datenbanken keine Historie führen, finden sich auch dort gewisse temporale Aspekte wieder: Zur Beschreibung jener zeitlichen Eigenschaften, die integraler Bestandteil der Entität der Realwelt sind, stellt das System temporale Datentypen zur Verfügung. Beispiele für solche zeitliche Eigenschaften sind das Geburtsdatum einer Person oder die maximale Ausleihfrist in einer Bibliotheksverwaltung. All diesen Zeitangaben ist aber gemeinsam, dass sie nichts über die Entwicklungsgeschichte der Entität der Realwelt aussagen, sondern lediglich eine weitere Beschreibung liefern. Sie stehen somit in keinem Bezug zur Historisierung (BALZER 2001).

### 4.2 Rollback-Datenbanken

In einer Rollback-Datenbank werden Datenwerte entsprechend der Transaktionszeit gespeichert. Sie hat eine Sequenz von Zuständen, die nach Transaktionszeit indiziert sind, d.h. nach der Zeit, zu der die Informationen in der Datenbank gespeichert wurden. Eine Rollback-Datenbank bewahrt die vergangenen Zustände der Datenbank, aber nicht die der realen Welt. Sie kann daher Fehler in vergangenen Informationen nicht korrigieren und kann auch nicht darstellen, dass einige Fakten für eine zukünftige Zeit gültig sind (htt). Zum Beispiel in einem Unternehmen das Gehalt der Mitarbeiter kann mit eine Transaktionszeit gespeichert werden.

| Customer | Amount | Name | Transaction Start Time | Transaction End Time |
|---|---|---|---|---|
| Cu_52 | 11000 | Smith | 01-01-1990 | 31-03-1995 |
| Cu_52 | 15000 | Smith | 1-04-1995 | 31-12-2000 |
| Cu_52 | 20000 | Smith | 01-01-2000 | 31-06-2005 |
| Cu_53 | 10000 | Jones | 01-01-1990 | 31-12-1995 |
| Cu_53 | 15000 | Jones | 01-01-1996 | 31-12-2000 |

Abbildung 10

### 4.3 Historische Datenbanken

Historische Datenbanken unterstützen nur die Gültigkeitszeit. Wie zuvor definiert, ist die Gültigkeitszeit eines Fakts die Zeit, zu der der Fakt wahr ist. Die Semantik der Gültigkeitszeit steht in en-

[4] http://ijrar.com/upload_issue/ijrar_issue_1060.pdf

ger Beziehung zur Realität. Historische Datenbanken können jedoch die Datenbank nicht so anzeigen, wie sie zu einem bestimmten Zeitpunkt in der Vergangenheit war.

| Emp_no | Salary | Name | Valid Start Time | Valid End Time |
|---|---|---|---|---|
| Em_52 | 11000 | Smith | 01-01-1990 | 31-03-1995 |
| Em_52 | 15000 | Smith | 1-04-1995 | 31-12-2000 |
| Em_52 | 20000 | Smith | 01-01-2000 | 31-06-2005 |
| Em_53 | 10000 | Jones | 01-01-1990 | 31-12-1995 |
| Em_53 | 15000 | Jones | 01-01-1996 | 31-12-2000 |

Abbildung 11[5]

Mit anderen Worten, eine historische Datenbank kann aufzeichnen, dass ein bestimmter Fakt F vom 1.1.1996 bis 5.4.1998 gültig war, aber es wird nicht aufgezeichnet, wann diese Gültigkeitszeiten zur Datenbank hinzugefügt wurden. Daher ist es nicht möglich zu sagen, dass am 1.1.1997 aufgezeichnet wurde, dass Fakt F wahr war - vielleicht haben wir retrospektiv aufgezeichnet, wann Fakt F zu einem Zeitpunkt nach dem 1.1.1997 wahr war. Somit ist es möglich, darüber zu spekulieren, was wir für einen bestimmten Zeitpunkt für wahr halten, aber es ist nicht möglich, Fragen zu beantworten, wann wir wussten, dass die Fakten wahr sind, da keine Änderungen an der Datenbank aufgezeichnet wurden (htt).

### 4.4 Temporale Datenbanken(Bi-temporale)

Diese stellen sowohl Transaktionszeit als auch Gültigkeitszeit dar und sind daher sowohl historisch als auch rollback-fähig. Temporale Datenbanken erfassen Daten zur Gültigkeitszeit (z.B. dass wir glauben, dass Fakt F vom 1.1.1996 bis 5.4.1998 gültig ist) und die Transaktionszeit, zu der solche Daten in die Datenbank eingegeben wurden (z.B. dass wir diese Überzeugung am 6.4.1997 hinzugefügt haben). Dies bedeutet, dass wir jetzt unsere temporale Datenbank zurücksetzen können, um herauszufinden, was unsere Überzeugungen zur Gültigkeitszeit für jede vergangene Transaktionszeit waren (z.B. was haben wir am 3.2.1997 über Fakt F geglaubt?).

Die Vorteile beider Ansätze können kombiniert werden, indem sowohl die Transaktionszeit als auch die Gültigkeitszeit in der gleichen Relation unterstützt werden.Der Benutzer eines temporalen DBMS kann historische Informationen aus der Sicht eines früheren Zustands der Datenbank untersuchen; beide Arten von Zeit können in einer Abfrage manipuliert werden.

[5] http://ijrar.com/upload_issue/ijrar_issue_1060.pdf

In einer temporalen Datenbank können wir also explizit ein Szenario wie folgendes darstellen:

Am 17. Januar 1997 (Transaktionszeit) hat der Arzt im Patientenbericht vermerkt, dass der Patient am 12. Januar 1997 (Gültigkeitszeit) eine allergische Reaktion auf ein Sulfa-Medikament hatte und dass der Arzt glaubte, dass der Patient vom 20. Januar bis 20. Februar ein nicht-sulfa-haltiges Medikament einnehmen würde. Wenn der Arzt am 1. Februar entscheidet, dass der Patient das Medikament nicht mehr benötigt, wird die Datenbank geändert, um anzuzeigen, dass der Patient das nicht-sulfa-haltige Medikament nur vom 20. Januar bis 1. Februar eingenommen hat.

## 5. Temporale Datenbankmodellen

In den letzten Jahren wurden zahlreiche temporale Datenmodelle entwickelt. Temporale Datenmodelle werden in der Regel als Erweiterungen standardmäßiger nicht-temporaler Datenmodelle definiert, die Mittel zur Darstellung und Manipulation zeitabhängiger Daten bereitstellen.

Wichtigste Konzepte:

• Erweiterung der Tabellen um temporale Attribute

• Erweiterung der Anfragesprache um temporale Operationen

Temporale Datenmodelle meistens basieren auf dem relationalen Datenmodell, da es eine einfache Implementierung auf einem relationalen System ermöglicht. Im relationalen Datenmodell werden Tupel (Zeilen) mit Zeitstempeln versehen, und im objektorientierten Datenmodell werden Objekte (Attributwerte) mit Zeitstempeln versehen. Im relationalen Datenmodell werden Relationen (Tabellen) um zwei zusätzliche Attribute erweitert, von denen eines den Startzeitpunkt und das andere den Endzeitpunkt angibt, (C. S. Snodgrass 2009).

Es gibt aber auch vollständig neu entwickelte Modelle, wie das Temporale Datenmodell TDM von A Segev und A Shoshani. Dieses basiert auf Zeitsequenzen, die die Geschichte von Objekten repräsentieren und zusätzlich bestimmte temporale Eigenschaften wie Granularität oder Definitionsbereich der Gültigkeitszeit, besitzen. Temporale Datenmodelle unterscheiden sich weiterhin in den unterstützten Zeitdimensionen und in der Art der Repräsentation temporaler Daten.

**Das Datenmodell BCDM**

Ein Datenmodell ist ein formaler Rahmen zur Beschreibung von Datenstrukturen und Operationen auf Daten. Es wird angenommen, dass die abzuspeichernde Information mit Hilfe von Objekten und

Beziehungen zwischen Objekten modelliert werden kann. Zeitstempel werden im BCDM als temporale Elemente abgebildet, die sich aus einem oder mehreren bitemporalen Chroni (tt, tv) zusammensetzen; dabei symbolisiert tt eine Transaktionszeitangabe und tv eine Gültigkeitszeitangabe. Ein Chronon ist eine Zeitangabe, die den kleinsten in einem gegebenen Kontext betrachteten Zeitraum repräsentiert und nicht weiter zerlegt werden kann. Das Datenmodell BCDM (bitemporal conceptual data model) ist Teil und Grundlage des Sprachvorschlages TSQL2. Als konzeptionelles Datenmodell soll es die Semantik temporaler Daten möglichst einfach und deutlich darstellen können, ohne dabei besonderen Wert auf eine möglichst effiziente physikalische Realisierung legen zu müssen. Das BCDM ist ein erweitertes relationales Datenmodell, d.h. die Konzepte des relationalen Modells werden übernommen und erweitert (Hassani 2004).

| Name | Lohn | $V_s$ | $V_e$ |
|---|---|---|---|
| Meier | 1500 | 11 | 49 |
| Meier | 2000 | 50 | 54 |
| Meier | 1500 | 55 | 60 |
| Meier | 2500 | 61 | 67 |

a)

| Name | Lohn | V |
|---|---|---|
| Meier | 1500 | {11,...,49},{55,...,60} |
| Meier | 2000 | {50,...,54} |
| Meier | 2500 | {61,...,67} |

b)

Abbildung 12Temporale Relation mit Tupelstempelverfahren

a) in erster Normalform

b) Gemäß BCDM (nicht in erster Normalform)

In einer Gültigkeits- bzw. Transaktionszeitrelation existieren keine zwei Tupel, deren Werte für die Attribute (Name, Lohn) übereinstimmen. Wiederholt sich ein Zustand bzw. Ereignis wird kein neues Tupel erzeugt sondern der Zeitstempel des bereits vorhandenen Tupels entsprechend erweitert. Der Primarschlüssel muss nicht um die Gültigkeits- bzw. Transaktionszeit erweitert werden. In den temporalen Relationen des BCDM wird das Tupelstempelverfahren angewendet.

Zum Vergleich zeigt Abb.12.a eine Gültigkeitszeitzustandstabelle, die ebenfalls das Tupelstempelverfahren benutzt, aber vollständig in Erster Normal form ist. Die Gültigkeitszeitintervalle werden in den Attributen Vs und Ve durch ihre Anfangs- und Endpunkte beschrieben. Weil diese Attribute atomar sind, beinhaltet ein Tupel stets nur ein Zeitintervall. Wiederholt sich ein Zustand bzw. ein Ereignis in der jeweiligen Zeitdimension, muss ein neues Tupel eingefügt werden. Im gezeigten Beispiel ist das bei dem Gehalt von 1500 Mark der Fall. Der Primarschlüssel muss um die Gültigkeitszeit erweitert werden.

# 6. Operationen in temporalen Datenbanken mit SQL11

Folgende Kapitel ist an (Krishna Kulkarni, Jan-Eike Michels 2012) basiert.

## 6.1 Anwendungszeitraumstabellen

Anwendungsspezifische Zeitraumtabellen sind dafür vorgesehen, den Anforderungen von Anwendungen gerecht zu werden, die daran interessiert sind, Zeiträume zu erfassen, in denen die Daten in der realen Welt als gültig angesehen werden. Ein typisches Beispiel für solche Anwendungen ist eine Versicherungsanwendung, bei der es notwendig ist, die spezifischen Richtliniendetails eines bestimmten Kunden zu verfolgen, die zu einem bestimmten Zeitpunkt wirksam sind.

Eine Hauptanforderung solcher Anwendungen ist, dass der Benutzer für die Festlegung der Start- und Endzeiten des Gültigkeitszeitraums von Zeilen verantwortlich ist und der Benutzer frei wählen kann, beliebige Zeitwerte für die Start- und Endzeiten anzugeben, entweder in der Vergangenheit, Gegenwart oder Zukunft. Eine weitere Anforderung solcher Anwendungen ist, dass dem Benutzer gestattet wird, die Gültigkeitszeiträume der Zeilen zu aktualisieren, wenn Fehler entdeckt werden oder neue Informationen verfügbar sind.

Jede Tabelle, die eine Zeitraumsdefinition mit einem benutzerdefinierten Namen enthält, ist eine anwendungsspezifische Zeitraumtabelle.

```
CREATE TABLE Emp(

 ENo INTEGER,

EStart DATE,

EEnd DATE,

EDept INTEGER,

PERIOD FOR EPeriod (EStart, EEnd) )


INSERT INTO Emp

VALUES (22217, DATE ‘2010-01-01’, DATE '2011-11-12', 3)
```

| ENo | EStart | EEnd | EDept |
|---|---|---|---|
| 22217 | 2010-01-01 | 2011-11-12 | 3 |

UPDATE Emp

FOR PORTION OF EPeriod

FROM DATE '2011-02-03'

TO DATE '2011-09-10'

SET EDept = 4

WHERE ENo = 22217

| ENo | EStart | EEnd | EDept |
|---|---|---|---|
| 22217 | 2010-01-01 | 2011-02-03 | 3 |
| 22217 | 2011-02-03 | 2011-09-10 | 4 |
| 22217 | 2011-09-10 | 2011-11-12 | 3 |

DELETE Emp

FOR PORTION OF EPeriod

FROM DATE '2011-02-03'

TO DATE '2011-09-10' WHERE ENo = 22217

| ENo | EStart | EEnd | EDept |
|---|---|---|---|
| 22217 | 2010-01-01 | 2011-02-03 | 3 |
| 22217 | 2011-09-10 | 2011-11-12 | 3 |

Der letzte Abschnitt gab ein Beispiel für eine Emp-Tabelle, bei der man erwarten würde, dass ENo der Primärschlüssel ist. Betrachtet man jedoch das Beispielergebnis der UPDATE-Anweisung, so gibt es drei Zeilen, die alle die ENo 22217 haben. Dieses Beispiel zeigt, dass der Primärschlüssel auch die Anwendungszeitraums-Spalten EStart und EEnd umfassen muss.

Das Hinzufügen von EStart und EEnd zum Primärschlüssel wird jedoch nicht ausreichen. Betrachten Sie die folgenden Daten: Die Tripel (22217, 2010-01-01, 2011-09-10) und (22217, 2010-02-03,

2011-11-12) sind keine Duplikate, sodass sie akzeptable Werte für einen herkömmlichen Primärschlüssel auf diesen drei Spalten wären. Beachten Sie jedoch, dass sich die Anwendungszeitperioden dieser Zeilen überschneiden. Semantisch bedeutet dies, dass der Mitarbeiter mit der ENo 22217 in den Abteilungen 3 und 4 während des Zeitraums vom 3. Februar 2010 bis zum 10. September 2011 tätig ist. Möglicherweise möchte der Benutzer einem Mitarbeiter erlauben, in zwei Abteilungen zu arbeiten. Die häufigere Anforderung besteht jedoch darin, dass ein Mitarbeiter zu einem bestimmten Zeitpunkt genau einer Abteilung angehört. Um dies zu erreichen, muss es möglich sein, überlappende Anwendungszeitperioden zu verbieten, was mit folgender Syntax angegeben werden kann:

```
ALTER TABLE Emp

ADD PRIMARY KEY (ENo, EPeriod WITHOUT OVERLAPS)
```

In SQL:2011 können Anwendungszeitperiodentabellen mit der regulären Abfragesyntax abgefragt werden. Um beispielsweise die Abteilung abzurufen, in der der Mitarbeiter 22217 am 2. Januar 2011 gearbeitet hat, kann die Abfrage wie folgt ausgedrückt werden:

```
SELECT Name, Edept

FROM Emp WHERE ENo = 22217

AND EStart <= DATE '2011-01-02'

AND EEnd > DATE '2011-01-02'
```

Eine einfachere Möglichkeit, die obige Abfrage zu formulieren, besteht darin, eines der Periodenprädikate zu verwenden, die in SQL:2011 für die Bedingungen mit Perioden zur Verfügung gestellt werden: CONTAINS, OVERLAPS, EQUALS, PRECEDES, SUCCEEDS, IMMEDIATELY PRECEDES und IMMEDIATELY SUCCEEDS. Die obige Abfrage könnte beispielsweise auch mit dem Prädikat CONTAINS ausgedrückt werden:

```
SELECT Ename, Edept

FROM Emp

WHERE ENo = 22217

 AND EPeriod CONTAINS DATE '2011-01-02'
```

Möchte man alle Abteilungen wissen, in denen der Mitarbeiter mit der Nummer 22217 im Zeitraum vom 1. Januar 2010 bis 1. Januar 2011 gearbeitet hat, könnte man die Abfrage wie folgt formulieren:

```
SELECT Ename, Edept

FROM Emp

WHERE ENo = 22217

AND EStart < DATE '2011-01-01'

AND EEnd > DATE '2010-01-01';
```

In dieser Abfrage wird das geschlossene-offene Modell verwendet, d.h., der Zeitraum umfasst den 1. Januar 2010, aber schließt den 1. Januar 2011 aus. Alternativ kann dieselbe Abfrage mit dem OVERLAPS-Prädikat wie folgt ausgedrückt werden:

```
SELECT Ename, Edept

FROM Emp

WHERE ENo = 22217 AND

EPeriod OVERLAPS

PERIOD (DATE '2010-01-01',

DATE '2011-01-01');
```

### 6.2 System versionierte Tabellen

System-versioned Tabellen sind dazu gedacht, den Anforderungen von Anwendungen gerecht zu werden, die aus geschäftlichen oder rechtlichen Gründen oder beidem eine genaue Historie von Datenänderungen aufrechterhalten müssen. Ein typisches Beispiel für solche Anwendungen ist eine Banking-Anwendung, bei der es notwendig ist, frühere Zustände von Kundendaten zu speichern, damit Kunden eine detaillierte Historie ihrer Konten erhalten können. Es gibt auch viele Beispiele, bei denen bestimmte Institutionen gesetzlich verpflichtet sind, historische Daten für einen bestimmten Zeitraum aufzubewahren, um regulatorische und Compliance-Anforderungen zu erfüllen.

Eine wichtige Anforderung solcher Anwendungen ist, dass jedes Update oder Löschen einer Zeile automatisch den alten Zustand der Zeile vor dem Update oder Löschen aufbewahrt. Eine weitere wichtige Anforderung ist, dass das System und nicht der Benutzer die Start- und Endzeiten der Zei-

ten für die Zeilen verwaltet und dass Benutzer den Inhalt historischer Zeilen oder die mit den Zeilen verbundenen Zeiträume nicht ändern können. Jegliche Aktualisierungen der Zeiträume von Zeilen in einer system-versioned Tabelle müssen nur durch das System als Ergebnis von Aktualisierungen der Nicht-Zeit-Spalten der Tabelle oder als Ergebnis von Zeilenlöschungen durchgeführt werden. Dies gewährleistet, dass die aufgezeichnete Historie von Datenänderungen nicht manipuliert werden kann, was entscheidend ist, um Auditing- und Compliance-Vorschriften zu erfüllen.

Jede Tabelle, die eine Periodendefinition mit dem standardisierten Namen "SYSTEM_TIME" enthält und die Schlüsselwörter "WITH SYSTEM VERSIONING" in ihrer Definition enthält, ist eine system-versioned Tabelle. Ähnlich wie bei application-time period Tabellen können Benutzer beliebige Namen für die Spalten wählen, die als Start- und Endspalten der SYSTEM_TIME-Periode fungieren. Obwohl SQL:2011 zulässt, dass die Datentypen der Start- und Endspalten entweder DATE oder ein Timestamp-Typ sein können (solange die Datentypen beider Spalten identisch sind), werden in der Praxis die meisten Implementierungen den Timestamp-Typ mit der höchsten Genauigkeit für Bruchteile von Sekunden als Datentyp für die Start- und Endspalten der System-Time-Periode bereitstellen. Zum Beispiel:

```
CREATE TABLE Emp (
  ENo INTEGER,
  Sys_start TIMESTAMP(12) GENERATED ALWAYS AS ROW START,
  Sys_end TIMESTAMP(12) GENERATED ALWAYS AS ROW END,
  EName VARCHAR(30),
  PERIOD FOR SYSTEM_TIME (Sys_start, Sys_end)
) WITH SYSTEM VERSIONING
```

Ähnlich wie bei Anwendungszonen verwenden Systemzeitperioden das geschlossene-geöffnete Periodenmodell. Zu jedem Zeitpunkt wird eine Zeile in einer systemversionierten Tabelle als aktuelle Systemzeile betrachtet, wenn der Systemzeitraum dieser Zeile die aktuelle Zeit enthält. Eine Zeile, die keine aktuelle Systemzeile ist, wird als historische Systemzeile betrachtet.

Systemversionierte Tabellen unterscheiden sich von Anwendungszeitperiodentabellen in folgenden Punkten:

1) Im Gegensatz zu Anwendungszeitperiodentabellen dürfen Benutzer den Werten der Spalten Sys_start oder Sys_end nicht zuweisen oder sie ändern. Diese Werte werden automatisch vom Datenbanksystem zugewiesen (und geändert). Aus diesem Grund müssen die Definitionen der Spalten Sys_start oder Sys_end die Schlüsselwörter GENERATED ALWAYS enthalten.

2) Bei einem INSERT in eine systemversionierte Tabelle wird der Wert der Spalte Sys_start automatisch auf den Transaktions-Zeitstempel gesetzt, einem speziellen Wert, der jeder Transaktion zugeordnet ist, und der Wert der Spalte Sys_end wird auf den höchsten Wert des Datentyps der Spalte gesetzt.

Angenommen, die folgende INSERT-Anweisung wird in einer Transaktion ausgeführt, deren Transaktions-Zeitstempel 2012-01-01 09:00:004 ist:

INSERT INTO Emp (ENo, EName)

VALUES (22217, 'Joe')

Die resultierende Tabelle sieht wie folgt aus (unter der Annahme, dass sie zuvor leer war):

| ENo | Sys_Start | Sys_End | EName |
|---|---|---|---|
| 22217 | 2012-01-01 09:00:00 | 9999-12-31 23:59:59 | Joe |

3) UPDATE und DELETE auf systemversionierte Tabellen wirken sich nur auf aktuelle Systemzeilen aus. Benutzer dürfen historische Systemzeilen nicht aktualisieren oder löschen. Benutzern ist es auch nicht gestattet, die Start- oder Endzeit des Systemzeitraums sowohl für aktuelle Systemzeilen als auch für historische Systemzeilen zu ändern.

4) UPDATE und DELETE auf systemversionierte Tabellen führen zur automatischen Einfügung einer historischen Systemzeile für jede aktualisierte oder gelöschte aktuelle Systemzeile.

Ein UPDATE-Statement auf einer systemversionierten Tabelle fügt zunächst eine Kopie der alten Zeile ein, wobei die Endzeit des Systemzeitraums auf den Transaktions-Zeitstempel gesetzt wird, um anzuzeigen, dass die Zeile ab dem Transaktions-Zeitstempel nicht mehr aktuell ist. Dann wird die Zeile aktualisiert, wobei die Startzeit des Systemzeitraums auf den Transaktions-Zeitstempel geändert wird, um anzuzeigen, dass die aktualisierte Zeile ab dem Transaktions-Zeitstempel die aktuelle Systemzeile ist.

Die folgende UPDATE-Anweisung ändert den Namen des Mitarbeiters mit der Nummer 22217 von Joe zu Tom, und zwar ab dem Transaktions-Zeitstempel der Transaktion, in der die UPDATE-Anweisung ausgeführt wurde:

```
UPDATE Emp

SET EName = 'Tom'

WHERE ENo = 22217
```

Eine historische Systemzeile, die dem Zustand der Zeile vor dem Update entspricht, wird zuerst eingefügt und dann wird das Update durchgeführt. Angenommen, die obige Anweisung wird in einer Transaktion mit dem Transaktions-Zeitstempel 2012-02-03 10:00:00 ausgeführt, dann ergibt sich das folgende Ergebnis:

| ENo | Sys_Start | Sys_End | EName |
|---|---|---|---|
| 22217 | 2012-01-01 09:00:00 | 2012-02-03 10:00:00 | Joe |
| 22217 | 2012-02-03 10:00:00 | 9999-12-31 23:59:59 | Tom |

Ein DELETE-Statement auf einer System-Versionstabelle löscht die qualifizierenden Zeilen tatsächlich nicht. Stattdessen ändert es die Systemzeit-Endzeit dieser Zeilen auf den Transaktions-Zeitstempel, um anzuzeigen, dass diese Zeilen ab dem Transaktions-Zeitstempel nicht mehr aktuell sind.

Das folgende DELETE-Statement ändert einfach die Systemzeit-Endzeit der aktuellen Systemzeile für den Mitarbeiter mit der Nummer 22217 auf den Transaktions-Zeitstempel der Transaktion, in der das DELETE-Statement ausgeführt wurde:

DELETE FROM Emp

WHERE ENo = 22217

Angenommen, das obige Statement wird in einer Transaktion mit dem Transaktions-Zeitstempel 2012-06-01 00:00:00 ausgeführt, dann ergibt sich folgende Zeile als Endergebnis:

| ENo | EStart | EEnd | EName |
|---|---|---|---|
| 22217 | 2012-01-01 09:00:00 | 2012-06-01 00:00:00 | Joe |

Die Durchsetzung von Einschränkungen bei System-Versionstabellen ist im Vergleich zu Anwendungszeitbereichstabellen einfacher, da Einschränkungen nur für aktuelle Systemzeilen durchgesetzt werden müssen. Historische Systemzeilen dienen als unveränderliche Momentaufnahmen der Vergangenheit, wodurch die Notwendigkeit entfällt, Einschränkungen für sie durchzusetzen. Folglich ist es nicht erforderlich, die Systemzeitraum-Start- und -Endspalten oder den Periodennamen in der Definition von Primärschlüssel- und referenziellen Einschränkungen bei System-Versionstabellen einzubeziehen.

SQL:2011 bietet drei syntaktische Erweiterungen für diesen speziellen Zweck. Diese sind nur in Abfragen auf system-versioned Tabellen erlaubt.

Die erste Erweiterung ist die FOR SYSTEM_TIME AS

OF Syntax, die nützlich ist, um den Tabelleninhalt zu einem bestimmten Zeitpunkt abzufragen.

SELECT ENo,EName,Sys_Start,Sys_End

FROM Emp FOR SYSTEM_TIME

FROM TIMESTAMP '2011-01-02 00:00:00’

TO TIMESTAMP '2011-12-31 00:00:00'

Beachten Sie, dass der in (FROM ... TO ...) angegebene Zeitraum dem closed-open-Periodenmodell entspricht, während der in (BETWEEN ... AND ...) angegebene Zeitraum dem closed-closed-Periodenmodell entspricht.

SELECT ENo,EName,Sys_Start,Sys_End

FROM Emp FOR SYSTEM_TIME

BETWEEN TIMESTAMP '2011-01-02 00:00:00'

AND TIMESTAMP '2011-12-31 00:00:00'

Wenn eine Abfrage auf System-Versionstabellen keine der oben genannten drei syntaktischen Optionen angibt, wird standardmäßig angenommen, dass die Abfrage "FOR SYSTEM_TIME AS OF CURRENT_TIMESTAMP" verwendet und nur die aktuellen Systemzeilen als Ergebnis zurückgegeben werden. Zum Beispiel gibt die folgende Abfrage nur die aktuellen Systemzeilen der Tabelle Emp zurück:

SELECT ENo, EName, Sys_Start, Sys_End

FROM Emp

Um sowohl aktuelle als auch historische Systemzeilen einer System-Versionstabelle abzurufen, kann eine Abfrage wie folgt verwendet werden:

SELECT ENo, EName, Sys_Start, Sys_End

FROM Emp FOR SYSTEM_TIME FROM

TIMESTAMP '0001-01-01 00:00:00' TO

TIMESTAMP '9999-12-31 23:59:59'

### 6.3 Bi-temporaleTabellen

Eine Tabelle kann sowohl eine system-versioned-Tabelle als auch eine application-time period-Tabelle sein. Zum Beispiel:

CREATE TABLE Emp(

ENo INTEGER,

EStart DATE,

EEnd DATE,

EDept INTEGER,

PERIOD FOR EPeriod (EStart, EEnd),

```
Sys_start TIMESTAMP(12) GENERATED ALWAYS AS ROW START,

Sys_end TIMESTAMP(12) GENERATED ALWAYS AS ROW END,

EName VARCHAR(30),

PERIOD FOR SYSTEM_TIME (Sys_start, Sys_end),

PRIMARY KEY (ENo, EPeriod WITHOUT OVERLAPS),

FOREIGN KEY (Edept, PERIOD EPeriod) REFERENCES Dept (DNo, PERIOD DPeriod)

) WITH SYSTEM VERSIONING
```

Bitemporale Tabellen kombinieren die Möglichkeiten von System-Versioned- und Application-Time-Period-Tabellen. Ähnlich wie bei Application-Time-Period-Tabellen ist der Benutzer für die Angabe der Werte der Application-Time-Period-Start- und -Endspalten verantwortlich. Ähnlich wie bei System-Versioned-Tabellen wird bei einem INSERT in eine solche Tabelle automatisch der Wert der Systemzeit-Perioden-Startspalte auf den Transaktionszeitstempel und der Wert der Systemzeit-Perioden-Endspalte auf den höchsten Wert des Datentyps der Spalte gesetzt.

# 7. References


kein Datum. https://www.cs.uct.ac.za/mit_notes/database/htmls/chp18.html.

BALZER, STEPHANIE. 2001. „ENTWICKLUNG EINES METADATENGESTEUERTEN.“ Zugriff am 06 2023. https://www.cs.cmu.edu/~balzers/publications/metadatengesteuertes_historisierungswerkzeug.pdf.

C.J. Date, Hugh Darwen, Nikos Lorentzos. 2002. *Temporal Data & the Relational Model.* Herausgeber: 1. https://books.google.de/books?hl=en&lr=&id=O5J39gk63BYC&oi=fnd&pg=PP2&dq=temporal+Data+%26+the+Relational+Model%22+von+C.J.+Date&ots=omibu05N_5&sig=cSHUw7mAYd7dHJ6RXCfZCGuhWvY&redir_esc=y#v=onepage&q&f=false.

Ekta, Ankit Sharma, Chinu. 2018. „Temporal Databases – A Review.“ (IJRAR- International Journal of Research and Analytical Reviews). Zugriff am 2023. http://ijrar.com/upload_issue/ijrar_issue_1060.pdf.

Hassani, Taoufik Saissi. 2004. „Konzepte temporaler Datenbanken.“ Zugriff am 06 2023. https://homepages.thm.de/~hg10013/Lehre/MMS/SS04/hasani/text.htm.

Jensen, Christian & Snodgrass, Richard. 1999. *Temporal data management. Knowledge and Data Engineering.*

Krishna Kulkarni, Jan-Eike Michels. 2012. „Temporal features in SQL:2011.“ (IBM Corporation). Zugriff am 06 2023. https://dbs.uni-leipzig.de/file/Temporal%20features%20in%20SQL2011.pdf.

Mutschler, Luka. 2019. „Temporale Datenbanken.“ Fakultät für Informatik und Medien,HTWK Leipzig. Zugriff am 06 2013. https://www.imn.htwk-leipzig.de/~kudrass/Lehrmaterial/Oberseminar/2019/08-Abstract-TemporaleDB.pdf.

Osipov, Maxim. 2017. „Temporale Datenbanken.“ HOCHSCHULE FÜR TECHNIK, WIRTSCHAFT UND KULTUR. https://www.imn.htwk-leipzig.de/~kudrass/Lehrmaterial/Oberseminar/2017/10-Abstract-TemporaleDB.pdf.

Richard Snodgrass, llsoo Ahn. 1986. „Temporal Databases .“ *Computer.*

Sam, Dahlia. 2020. „What Is A Temporal Database?“ (My Tec Bits). https://www.mytecbits.com/oracle/oracle-database/what-is-temporal-database.

Snodgrass, Christian S. Jensen & Richard T. 2009. „Temporal Query Languages.“ In *Encyclopedia of Database Systems*.

Snodgrass, R.T. 2000. *Developing Time-Oriented Database Applications in SQL.* Morgan Kaufmann Publishers.

kein Datum. „Temporal database.“ (Temporal database - Information Science - 3077). Zugriff am 03. 06 2023. http://kuliahkaryawan.widyakartika.ac.id/IT/en/1-3077-2963/temporal-database_22001_kuliahkaryawan-widyakartika.html.

2023. „Temporal tables.“ 03. 01. https://learn.microsoft.com/en-us/sql/relational-databases/tables/temporal-tables?view=sql-server-ver15#how-do-i-query-temporal-data.